



interCEde

Designing a Maintainable and Testable Resource Access Layer
for Heterogeneous Computing Resources
Test Plan



Alexander Auer
Student Number: s1152698

CERN LHCb Experiment DiracX
Hogeschool Leiden
Informatica

Internship Period: 01-09-2026 –
28-02-2027
Version: 1.0.0
Version Date: October 7, 2026



Version history

Table 1: Version history of the test plan.

Version	Date	Author	Description
0.1.0	29-09-2026	Alexander Auer	First version: scope, test levels, environment, test cases and traceability.
0.2.0	07-10-2026	Alexander Auer	Contract suite also runs against the fake backend; test cases regrouped and refined; traceability, schedule and risks updated.

Contents

Version history	1
1 Introduction	3
1.1 Purpose	3
1.2 Relation to the other documents	3
1.3 Basis	3
1.4 Document structure	3
2 Test items and scope	4
2.1 In scope	4
2.2 Out of scope	4
3 Test levels	5
3.1 Static checks	5
3.2 Unit tests	5
3.3 Contract suite	5
3.4 Version tests	6
4 Test environment	7
4.1 Continuous integration	7
4.2 Stacks	7
4.3 Versions	7
5 Test cases	8
6 Traceability	11
7 Entry and exit criteria	12
7.1 Pull requests	12
7.2 Stacks	12
7.3 Backends	12
8 Reporting	13
9 Schedule	14
10 Risks	15
A References	16

1 Introduction

1.1 Purpose

This test plan describes how InterCEde is tested: what is tested and what is not, at which levels, in which environment, which test cases verify which requirements, and when a test run – and a backend as a whole – counts as passed.

1.2 Relation to the other documents

The Design Document explains *why* InterCEde is testable: structural protocols that can be replaced by fakes, and containerised resource systems for testing against the real thing (Auer, 2026a, §11). This test plan builds on that and makes it operational. The requirements it verifies are those of the Design Document (Auer, 2026a, §3).

Testing and evaluation are kept apart. This plan verifies that the *implementation* behaves as specified. The separate evaluation document assesses whether the *design* meets the design criteria DC-01 to DC-07 from the research (Auer, 2026b, §4). The two meet in one place: test results serve as evidence in the evaluation.

1.3 Basis

The test infrastructure described here follows the integration test scaffolding in the InterCEde repository (DIRACGrid, 2026). Where this plan refers to workflows, markers or fixtures, they are the ones defined there. Terminology follows the glossary of the Design Document.

1.4 Document structure

Section 2 defines what is and is not tested. Section 3 describes the test levels and Section 4 the environment they run in. Section 5 lists the test cases and Section 6 links them to the requirements. Section 7 states when a test run, a stack and a backend count as passed, Section 8 how results are reported, Section 9 when each part is built, and Section 10 the risks to testing.

2 Test items and scope

2.1 In scope

- The public API: the capability protocols, the data types, the registry and its configuration models, the credential types and the exception hierarchy (Auer, 2026a, §4.2).
- Every backend InterCEde ships, per resource system family: the composed backend with its transports and schedulers, the Monolithic backends and the Provider backends.
- The rules of the contract that only a real resource system can confirm, such as repeatable output collection and distinct per-copy secrets (Section 5).
- Behaviour across the supported versions of each resource system.
- The test infrastructure itself: the stack manifest and the layout of each stack.

2.2 Out of scope

- The payload: InterCEde never inspects what a job runs, so its behaviour is not tested.
- The consumer's own code, such as DiracX's scheduling and throttling policy. InterCEde only offers the mechanism.
- Multi-node scheduling. The stacks run single-node resource systems; the contract ends at the scheduler interface.
- Performance and load testing.
- Worker-node execution, orchestration, storage and catalogues, which are outside InterCEde altogether (Auer, 2026a, §14.1).

3 Test levels

InterCEde is tested at four levels. Each catches a kind of fault the others cannot.

3.1 Static checks

Every change is checked with a linter and formatter (ruff) and a strict type checker (mypy). The type checker is also a test in its own right: backends conform to the protocols structurally, without inheriting from them, so each backend module assigns its class to every protocol it implements. If the backend drifts from the contract, including a method that is not asynchronous, the type check fails (NFR3, NFR8).

3.2 Unit tests

Unit tests run without any external system. They cover what can be decided in isolation: validation of the data types, the registry, credential refresh, secret handling, command building and status parsing in schedulers, and the public API. The tests run with pytest in asynchronous mode, on Python 3.11, 3.12, 3.13 and 3.14.

The unit tests use an in-memory *fake backend* that implements every capability. Its constructor options and test hooks let a test refuse copies, fail single jobs, make the whole resource unavailable, finish a job with given output, or switch off container and resource-side staging support. Because the protocols are structural, consumer code and tests can use it without any resource system (NFR3).

3.3 Contract suite

The contract suite is one set of tests written only against the public API. It runs at two levels:

- **Against the fake backend**, as part of the unit tests on every pull request. This keeps the suite itself working before any real backend exists.
- **Against every real backend** in a containerised *stack*. Fakes always agree with the protocol, so the faults InterCEde exists to absorb only appear here.

Optional capabilities are checked the way consumers check them, with `isinstance` on the protocol; a test for a capability the backend lacks is skipped, not failed. A test that applies to only some backends, or is known to fail on one, is marked `backend_only` or `xfail_backend`. Behaviour that needs a job to finish or a failure to be injected is driven through the fake’s test hooks at unit level; against a real backend the same rule is checked on a real job.

In the stacks, each stack selects its subset of the suite with a pytest marker expression (DIRACGrid, 2026). Table 2 lists the markers.

Table 2: Markers selecting the applicable part of the contract suite.

Marker	Meaning
<code>remote</code>	Drives a Monolithic backend (a grid gateway: ARC, HTCondor-CE)
<code>scheduler</code>	Drives a composed backend (transport and scheduler)
<code>arc</code>	Requires an ARC CE
<code>htcondor</code>	Requires an HTCondor schedd or HTCondor-CE

Marker	Meaning
slurm	Requires a Slurm cluster
ssh	Requires a batch host reachable over SSH

3.4 Version tests

Each backend declares a support window: the oldest version sites still run, normally a long-term support release, and the newest release. The contract suite runs against the versions in that window, as a matrix axis next to the stack and its configuration (Auer, 2026a, §11.3). ARC is supported from version 7 only.

4 Test environment

4.1 Continuous integration

Two GitHub Actions workflows run the tests (DIRACGrid, 2026):

- **CI** runs on every pull request and every push to **main**: the pre-commit hooks (formatting, linting, type checking, commit message conventions) and the unit tests on Python 3.11, 3.12, 3.13 and 3.14.
- **Integration tests** generates a matrix of stack $\times$ configuration $\times$ version from the stack manifest, and runs each combination as its own job: it starts the stack, waits for its health checks, runs the stack's subset of the contract suite and uploads the daemon logs if anything fails.

The development environment and dependencies are managed with pixi.

4.2 Stacks

A stack is a self-contained resource system environment, described by one compose file and one or more configurations. The stack manifest is the single source of truth for which stacks, configurations and versions run. Adding a stack, a configuration or a version is one manifest entry plus files, without changes to the workflows. Table 3 lists the stacks currently in the manifest.

Table 3: Stacks in the manifest.

Stack	Versions	Markers	Notes
arc-slurm	latest	remote, arc	ARC CE in front of Slurm
htcondor	LTS, latest	remote, htcondor	HTCondor-CE in front of an HTCondor pool
ssh-slurm	latest	scheduler, slurm	Slurm reached over SSH
local-slurm	latest	scheduler, slurm, not ssh	Reuses ssh-slurm ; tests run inside the container

Every stack follows the same contract: it can be started locally with one command, every service has a health check, daemons run in the foreground and log to standard output, images and packages are pinned to exact versions, and credentials are generated when the stack starts and never committed. Because of this, everything CI does can be reproduced on a developer's machine, and the stacks double as development environments.

4.3 Versions

All versions are pinned. Renovate proposes updates as pull requests, grouped per resource system, so a new upstream release that breaks the contract appears as a failing, attributable pull request. Only the newest version is tracked automatically; older versions in a support window are moved by a deliberate decision.

5 Test cases

Test cases are identified by level: **S** for static checks, **U** for unit tests, **C** for the contract suite and **V** for version tests. The **C** cases run against the fake backend and against every real backend (Section 3). A contract test that needs an optional capability names it in brackets and is skipped on backends without it. In the last column, section numbers (§) and decisions (D) refer to the Design Document. The set grows with the implementation.

Table 4: Test cases.

ID	Test	Expected result	Covers
<i>Static checks</i>			
S1	Type-check each backend class against JobBackend and every optional protocol it implements.	Type check passes.	FR5, NFR3, NFR8
<i>Unit tests</i>			
U1	Resolve a valid configuration and provider through the registry.	A backend satisfying JobBackend .	FR1
U2	Resolve a configuration with an unknown kind or a missing field.	ConfigurationError naming the field.	FR1, NFR5
U3	Serialise a JobHandle and restore it; restore one with an unknown field or a newer format version.	Identical handle; unknown or newer handles rejected.	NFR2
U4	Use the same job ID on two resources as keys of a result map.	Two distinct handles.	FR3
U5	Build data types with invalid values: file names that leave the working directory, non-positive resources, a copy both submitted and refused, a failed outcome without a reason.	Rejected when the value is built.	FR3, §10.5
U6	Run an operation with missing, fresh and nearly expired credentials.	Provider called only when missing or nearly expired, with the operation's token scopes.	FR10
U7	Print, log and serialise per-copy secrets and credentials.	No secret material appears; the value is readable only on purpose.	NFR7
U8	Build a scheduler's submit command for copies with secrets.	Secrets absent from the job description.	FR2, NFR7
U9	Parse scheduler status output that lacks one of the requested IDs.	Common states; the missing ID reported as unknown.	FR4
U10	Make the resource unavailable during an operation.	ResourceUnavailableError for the whole operation.	FR3, D5
U11	Collect output whose members point outside the destination, or exceed a collection limit.	Failed outcome for that job; nothing written outside the destination.	§10.5

ID	Test	Expected result	Covers
U12	Check capabilities with isinstance on a backend with every capability, one with only the essential ones, and one without a lifecycle.	Each matches exactly the protocols it implements; without a lifecycle it is no JobBackend .	FR5, NFR3
U13	Import every name in the public API.	All names import; internal protocols such as Transport and Scheduler are not exported.	NFR6
U14	Validate the stack manifest and stack layout.	Schema and layout correct.	NFR4
<i>Contract suite</i>			
C1	Submit one specification as n copies, some of which are refused.	Every copy index appears once, under its handle or its refusal; handles are distinct.	FR2, FR3
C2	Submit copies with different identifiers and secrets.	Each job sees its own identifier and secret file, with restrictive permissions.	FR2, NFR7
C3	Delete a local input file right after submission.	The job is unaffected.	§5.1
C4	Query the status of submitted jobs, of an unknown ID and of a job on another resource.	One status per handle; the last two reported as unknown.	FR4
C5	Collect the output of a completed job twice (OutputRetriever).	Both collections succeed, match and lie under the destination.	FR8, D8
C6	Run a payload that writes files named like the output streams (OutputRetriever).	Streams, scheduler log and payload files kept apart.	FR8
C7	Kill, collect and read diagnostics for a mix of known and unknown jobs, where supported.	One outcome per handle; unknown jobs fail with a reason, nothing is raised.	FR3, D5
C8	Kill a running job (Killable).	Successful outcome; the job stops.	FR5
C9	Purge a completed job (Purgeable).	Successful outcome; afterwards reported as unknown.	FR5
C10	Read diagnostics from a running job twice (Diagnosable).	Same content both times; the job keeps running.	FR5

ID	Test	Expected result	Covers
C11	Submit what the backend cannot honour: outputs kept for collection where files cannot be kept, inputs or outputs the resource must move itself, a container image, or CPU or memory limits on the direct scheduler without cgroup v2.	Specification RejectedError ; nothing submitted.	FR5–FR7, D10, D11
C12	Run an operation with rejected credentials.	AuthenticationError for the whole operation.	FR3, FR10
C13	Connect over SSH to a host whose key is not known (ssh).	The whole operation fails.	§10.3
C14	Close a backend twice, then submit.	The second close is harmless; the submission raises.	§6.4
<i>Version tests</i>			
V1	Run the contract suite on every version in a backend’s support window.	Same results on every version.	NFR4

6 Traceability

Table 5 shows which test cases verify each requirement of the Design Document (Auer, 2026a, §3). Every requirement is covered by at least one test case, except NFR1: that InterCEde never issues credentials is a property of what the code does *not* contain, and is therefore checked in code review rather than by a test.

Table 5: Requirements and the test cases that verify them.

Req.	Requirement (short)	Test cases
FR1	Backend from the registry	U1, U2
FR2	Many copies, own environment and secrets	U8, C1, C2
FR3	Result per job	U4, U5, U10, C1, C7, C12
FR4	Status for any job, unknown never dropped	U9, C4
FR5	Optional capabilities	S1, U12, C8, C9, C10, C11
FR6	Who moves the data; refusal at submission	C11
FR7	Container images	C11
FR8	Repeatable retrieval, separate streams	C5, C6
FR9	All three families	Contract suite on a stack per family (Table 3)
FR10	Several credential kinds	U6, C12
NFR1	No credential issuance	Code review
NFR2	Agnostic and stateless	U3
NFR3	Structural contracts	S1, U12
NFR4	Tested against real systems	U14, V1
NFR5	Closed library	U2
NFR6	Small public API	U13
NFR7	Secret handling	U7, U8, C2
NFR8	Asynchronous	S1

7 Entry and exit criteria

7.1 Pull requests

A pull request can be merged when the pre-commit hooks and the unit tests, including the contract suite against the fake backend, pass on Python 3.11 to 3.14. The integration workflow is not yet a required check; until it is, a failing integration run does not block a merge (DIRACGrid, 2026).

7.2 Stacks

A stack can be used for testing once it follows the stack contract (Section 4), passes the manifest checks (U14) and has an entry in the manifest. A selected stack without a client configuration fails in CI instead of being skipped, so that a run in which every test was skipped can never appear green.

7.3 Backends

A backend is supported once its subset of the contract suite passes against its stack, on every version in its support window (Auer, 2026a, §8.7). Tests for capabilities the backend does not have are skipped, not failed.

8 Reporting

- **Pull requests:** every workflow and every matrix job reports its status on the pull request, named after its stack, configuration and version.
- **Failures:** when an integration job fails, the logs of all daemons in the stack are uploaded as an artifact, so a failure can be diagnosed without rerunning it.
- **Upstream changes:** version updates arrive as Renovate pull requests, so their test results are recorded like any other change.
- **Scheduled runs** (planned): a scheduled workflow will run the full matrix, including unpinned versions, and open an issue when an upstream pre-release breaks the contract. It cannot block a merge.
- **Evaluation:** test results are used as evidence in the separate evaluation document.

9 Schedule

Testing follows the implementation. The repository divides the test work into phases (DIRACGrid, 2026): in phase 0 stacks are validated with the resource systems' own tools, phase 1 provides the scaffolding, and phase 2 introduces the contract suite. Combined with the implementation order, this gives the following steps:

1. **Scaffolding** (phases 0 and 1): the stack manifest, the harness fixtures, the workflows and the first stacks, validated with the resource systems' own command-line tools, since the InterCEde protocols do not exist yet.
2. **Core and fake backend**: the core contract, the data structures and the fake backend, with their unit tests and the first version of the contract suite running against the fake.
3. **Contract suite in the stacks** (phase 2): the same suite runs in the stacks and replaces the native-tool tests.
4. **First real backend**: HTCondor-CE, tested against its stack.
5. **Further backends**: each with its own stack and manifest entry.

10 Risks

Table 6 lists the risks to testing. Risks to the project as a whole are covered in the internship report.

Table 6: Testing risks.

Risk	Impact	Mitigation
Green runs that tested nothing	Until the contract suite exists, the workflow treats “no tests selected” as a pass, so a stack can report green without verifying anything.	Remove this tolerance once the contract suite lands.
Integration tests not yet gating	A contract violation can be merged.	Make the integration workflow a required check.
Fake backend drifts from the contract	Unit tests pass against behaviour no real backend has.	The type checker proves the fake’s conformance (S1); the same contract suite runs against real backends.
Unreliable daemons	Intermittent failures erode trust in the results.	Health checks gate every stack; one job per stack isolates failures; daemon logs are kept.
Upstream packaging changes	Image builds break for reasons outside InterCEde.	Pinned versions; image builds kept separate from the test runs.
Single-node stacks	Multi-node scheduling is never exercised.	Accepted: the contract ends at the scheduler interface.
Growing CI time	Every stack, configuration and version adds jobs.	Pull requests run a representative subset; a scheduled run, still to be added, covers the full matrix.

A References

- Auer, A. (2026a). *Designing a maintainable and testable resource access layer for heterogeneous computing resources: Design document* (Design document). Hogeschool Leiden / CERN LHCb DiracX. https://github.com/Alexander-Auer/intercede/blob/main/docs/artifacts/pdfs/Design_document.pdf
- Auer, A. (2026b). *Designing a maintainable and testable resource access layer for heterogeneous computing resources: Research report* (Research report). Hogeschool Leiden / CERN LHCb DiracX. https://github.com/Alexander-Auer/intercede/blob/main/docs/artifacts/pdfs/Research_Report.pdf
- DIRACGrid. (2026). *Feat: Integration test scaffolding (pull request #25)*. Retrieved September 29, 2026, from <https://github.com/DIRACGrid/intercede/pull/25>